

Capstone Proposal: Wayfarer's Crawl

Enemy AI Focused Dungeon Crawler

By:

[REDACTED]

[REDACTED]

02/06/2026

Overview

Player engagement is a fundamental aspect of a game that game developers need to consider. One way to help maintain player engagement is through fresh experiences throughout the moment-to-moment gameplay. In single-player games, the challenge of keeping the players engaged is on the developers; how do you give the player new experiences every time they play without the unpredictability of human opponents? One approach to accomplish this is through the use of enemy Artificial Intelligence (AI) to generate this necessary challenge and variety. In an article written by Sergio Ocio, a current game developer at Infinity Ward studios, he mentions how enemy AI needs to have three primary goals: to provide feedback, to seem realistic, and to be consistent. When the player interacts with the enemy, the enemy AI needs to properly respond with a degree of realism, meaning their actions seem logically grounded in a consistent way as opposed to acting randomly (Barriaes).

Our game aims to create an enemy AI that both challenges the player and responds realistically to the player's actions. To do this, we will expand upon our current game, which is a 2D, action rogue-like game with multiple enemies that we began developing last semester. Rogue-likes are centered around resetting the player's progress upon death, which encourages the player to learn enemy behaviors and master the game's systems. In line with the goals of feedback, realism, and consistency as mentioned by Sergio Ocio, we will be adding new enemy types and design a boss encounter to test the player's skill. We will also introduce distinct player classes, giving the player multiple ways to approach the game's challenges.

Enemy AI

Currently in the game, we have three basic enemy types: a melee enemy, a ranged enemy, and a healer enemy. Each enemy uses a finite state machine to act as the enemy AI,

which determines what actions they take. The melee enemy is designed to quickly approach the player and attack them at close range, while the ranged enemy maintains distance and fires damaging projectiles. Both the melee and ranged enemies can retreat from combat and locate a nearby healer enemy once they have taken a certain amount of damage, after which the healer will heal them before they re-engage with the player. The healer enemy's role is to heal nearby allies at low health. When no allies nearby need healing, the healer can also act as a melee enemy.

The finite state machines used across all current enemies can be described as a variable or enum-based. This means that each enemy script (the code used for each enemy) contains a list of states, and the enemies check if the current state needs to be changed based on in-game events that take place, such as taking damage, a player getting too close, or a nearby enemy taking damage. The current implementation does suffer from several issues, those being future reusability and developing more complex enemy behaviors. To address these problems, our team is considering two solutions that require further research. Those solutions are to either refactor the current state machine or to implement behavioral trees.

Refactor State Machine

The first option we are considering to improve our enemy AI is to refactor the current enemy finite state machine to support a hierarchical structure. This would allow each state to inherit logic from a parent state. For example, the current Hurt state (which is the state an enemy enters after being damaged) could become a parent state for various "fury" states triggered by taking damage. A key advantage of this method is reducing repeated code in the enemy AI scripts. This would help with the first problem we are encountering, which is code reusability. One resource that highlights many of the strengths and weaknesses of hierarchical state machines is the book *Game Programming Patterns*, written by Robert Nystrom, a former employee at Electronic Arts, a large game development company. In this book, the author goes over a few examples of how this can be used to create multiple states with similar logic: "We

could define a class for an 'on ground' state that handles jumping and ducking. Standing, walking, running, and sliding would then inherit from that and add their own additional behavior” (Nystrom). However, this approach does not fully address our second concern, the ability to create complex enemy behaviors. One of the limitations of finite state machines is their ability to give structure to complex enemy AI. Finite state machines can be thought of as a web of interconnected states; while the addition of new states will add complexity, it will also introduce new problems as each new state requires transitions to and from existing states.

Implementing Behavior Trees

Our second option for developing complex enemies is the use of behavior trees, another method for developing enemy AI. If finite state machines can be thought of as a web of interconnected states, a behavior tree is structured like a tree with branching pathways for decision logic. Along each “branch”

Leaf Node:	These are low-level tasks along a behavior tree that define what the enemy will be doing. These nodes typically contain unique code to define a specific task, similar to a state that defines an enemy's behavior.
Action Leaf Node:	These types of leaf nodes execute enemy actions or methods. For example, this could be used by an enemy to attack the player.
Conditional Leaf Node:	These types of leaf nodes check the game world. For example, this could be used by an enemy to see if the player is close enough for them to attack.
Composite Node:	These nodes define the relationship between a set of child nodes alongside how they should be executed. These nodes rarely have unique code that differs between enemies, but rather are the scaffolding for a behavior tree.
Selector Composite Node:	These types of composite nodes will run the logic of the first successful child node. For example, an enemy may have a selector node with a child node that checks to see if a nearby ally needs healing. If not, it will move on to the second child node, which checks if there is a nearby player to attack.
Sequence Composite Node:	These types of composite nodes will run each child node if all of them can be successfully executed. For example, an enemy may have a sequence node that first requires them to check for a nearby ally who is a healer. If that child node was successful, it would attempt to execute the next one, which would involve the enemy moving towards the healer.
Decorator Composite Node:	These types of composite nodes act as a “wrapper” for a child node and modify how it behaves. For example, an enemy may have an action node for pathfinding, but a detector could be used to alter the logic of said action node.
Table 1: Behavior Tree Node Types	

of a behavior tree are different nodes. Some nodes function similarly to states, while others act more like state transitions. The types of nodes we would use are shown in Table 1, which outlines the different leaf and composite nodes. The implementation of behavior trees that use composite and leaf nodes will allow us to implement more complex enemy AI without the difficulty of managing an ever-growing number of state transitions.

Another benefit of using a behavior tree is the reusability of both the leaf and composite nodes. Within the paper *The Curation Tree: A Lightweight Behavior Tree Framework for Implementing Puzzle and Narrative Games*, written by three university researchers, the authors discuss how a behavior tree can simplify the development process of narrative and puzzle

games: “A challenge with any behavior tree approach is how to split the game mechanics across behaviors. Small behaviors are more easily reused, tested, modified, and customized. However, a large behavior can better manage complex logic by keeping it in one place” (Normoyle). For our project, composite nodes will only need to be developed once, as they form the core scaffolding of the behavioral tree, and many leaf node implementations can also be reused across multiple different enemy types.

Enemy Design Goals

New Enemy Types

Currently, the existing roster of enemies—the melee, ranged, and healer—is limited when it comes to its ability to support a diverse range of combat encounters. To address this, we plan to introduce new enemy types using more complex enemy AI than what is currently being used by our roster of enemies. These new enemies will be built using either a behavior tree to determine the actions they take or a hierarchical state machine with substates (a subclass of a parent state).

Teleporting “Assassin” Enemy: The goal of this enemy will be to teleport to the player, dodge player attacks, and react uniquely to the player if they are currently in the middle of an attack animation. Designed for rapid engagement and disengagement, it will use a teleport ability to close distance and evade. It will also feature a chance-based dodge mechanic to counter the player’s attacks, as well as a counterattack ability when the player misses an attack.

Dividing “Slime” Enemy: While testing the melee enemies, one of the prominent ways they can quickly defeat the player is by overwhelming them with numbers—a melee enemy may be easy to defeat, but five or ten can quickly become too much for the player to handle. Having an enemy that, upon death, will divide itself into two smaller and weaker versions of itself can help create more moments where the player may feel overwhelmed by enemies. There will be a

limit to how many times it can divide itself, to prevent an unending swarm of enemies.

Furthermore, this enemy will also have the ability to recombine if the player decides to retreat from the fight.

Knockback “Bruiser” Enemy: One issue with the current roster of enemies is that none of them interact with the player outside of dealing damage. A solution to this would be to add an enemy that is designed to knock the player and other enemies around, creating more chaotic combat scenarios. This, in tandem with ranged enemies, may cause the player to be knocked back into ranged projectiles. The goal of this enemy type will be to disperse all nearby entities (enemies and the player), alongside having attacks that will deal damage based on how fast the entity was moving before they collided with a surface or another entity.

First Boss Enemy

One of the primary goals of this game is to create a boss enemy, an enemy that is distinct from the rest of the enemies within the game, from both a design goal and an enemy AI perspective. This boss will use multiple behavior trees to act as the enemy AI model. The boss's enemy AI will move from one behavior tree to another as the fight progresses, with each tree being distinct from each other. These trees will correspond to different playstyles; for example, the first tree may have the boss acting passively, whereas the second will have the boss aggressively engaging with the player.

An initial concept for our first boss was one with charging attacks that would deal large amounts of damage and force the player to weave in and out of their attack range. After designing the first few enemies and learning that the game is more tailored for enemy-dense encounters, the first boss of this game will be based around the resurrection of enemies. Currently, each enemy has a state they enter when they die. This has so far only been used to signal to the enemy that they need to play their respective death animation and nothing more. However, with a boss enemy that interacts with enemies who are in this state, this could create an intense combat encounter with constant enemy density.

Resurrection “Necromancer” Boss: This boss will be designed as a combination of the current ranged enemy and the new teleporting enemy. This means they will have primarily ranged attacks while also having a focus on disengaging from the player. Their ranged projectile attacks will be much larger than the current ranged enemy’s projectiles, alongside potentially having the ability to split off into smaller projectiles upon colliding with a wall or surface. The boss will also possess the ability to resurrect enemies. This ability will likely be tied to a long attack animation that can be interrupted by the player to reward an aggressive playstyle. However, the boss will have certain resurrection abilities that cannot be interrupted, which will either be tied to the current health of the boss or the amount of time spent fighting the boss. Enemies resurrected by the boss may have some slight variations in their damage or health based on how difficult we wish to make this first boss.

Pathfinding

The enemy pathfinding system is capable of navigating toward the player while avoiding obstacles in a manner that demonstrates basic intelligent behavior. This system is used across multiple gameplay scenarios, including pursuing the player, retreating toward allied enemies, and wandering to randomized map locations. However, the current implementation contains persistent issues, most notably enemies becoming stuck within wall collision boundaries.

Potential solutions include migrating to a Navigation Mesh. A Navigation Mesh is a built-in Godot feature that defines walkable areas for enemy AI. While this approach may introduce additional edge cases, our current familiarity with the pathfinding system allows us to address them in a more efficient manner. The main focus for this change in the pathfinding is to improve wall awareness and decision-making logic, or introducing wall deadzones that force enemies to select pathing targets farther from obstacles. The deadzone approach, while simpler, introduces additional edge cases.

A demo will be applied using a Navigation Mesh to test the functionality and cost effectiveness of migrating the current pathfinding over into a Navigation Mesh. Capstone members will then decide if this is the correct choice of action as a team.

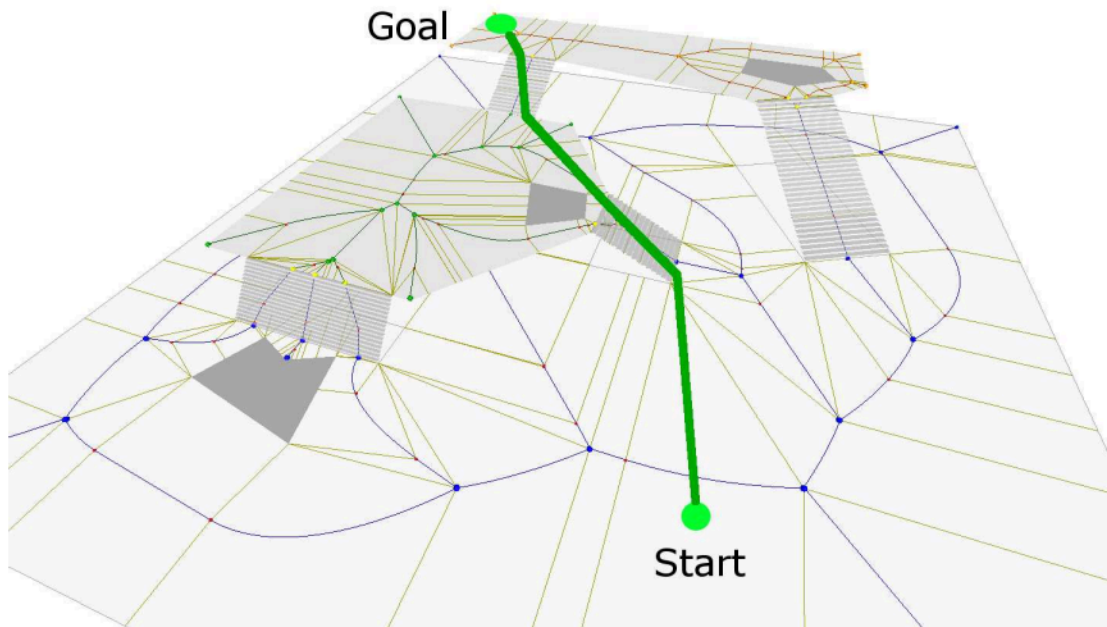


Figure 1: Navigation Mesh

(Navigation Meshes for Realistic Multi-Layered Environments)

Player Class

Players currently have access to a single playable class, the Samurai. This class is designed for aggressive, close-quarter melee combat, centered on three core attacks, light, heavy, and skill, each with distinct risk-reward dynamics. The player's movement is fluid, offering full 360-degree control and a sprint function. The current implementation also includes death and revive animation sequences, stamina burn and recovery, and damage visual effects.

In the upcoming semester, we will refactor the player class structure to support multiple character classes using an inheritance-based design. Currently, all functionality is managed within a single, large script built specifically for the Samurai. Adding new features or character

types risks conflicts with the existing Samurai specific logic. To solve this, our new design will implement a class hierarchy. A base 'Character' class (the parent) will handle shared logic such as global states, input processing, and fundamental actions like movement and stamina. Individual character classes like the Samurai (the children) will then inherit from this base class and implement their own unique scripts to define specific abilities and combat logic. This inheritance model enables clean, modular development, allowing us to cleanly separate the Samurai's melee functions from a future ranged class's projectile mechanics.

Our next development priority is to introduce a dedicated Ranged Archer class to diversify combat options. This class will specialize in zoning and area denial, controlling key positions and limiting enemy movement from a distance. It will Feature light and heavy bow attacks and a high-damage skill shot. For close-range emergencies, it will also include a weak melee attack, supported by several new animations. Additionally, if development time permits, we plan to introduce one additional class to further expand our game's variety. We are currently deciding between two concepts:

- A Shield-Bearing Tank, focused on high defense and crowd control, trading damage output for survivability. They would center on a shield bash to knock enemies back and a timed counterattack.
- A Pyromancer, a spellcaster designed for area-of-effect damage and fire effects.

To further deepen variety and character differentiation, we plan to implement a status effect system tied to each class's core identity. Each class will inflict a unique condition on enemies, the Samurai will apply Bleed, causing damage over time, the Ranged Archer will inflict Stun, temporarily immobilizing targets, the Tank (if developed) will specialize in Knockback, forcibly repositioning enemies to control their own space and allow time to breathe, and the Pyromancer (if developed) would introduce Burn, a dynamic effect that sets enemies ablaze for damage over time and can also create environmental fire hazards. A key element of Burn is its potential to disrupt enemy AI, causing afflicted targets to panic and move erratically until the

effect ends or they are defeated. These effects are designed to reward players for leaning on their classes' unique strengths, whether it's whittling down enemies, creating enough space to safely attack, or managing enemy positioning.

GUI

Our current GUI implementation provides a functional foundation. The core gameplay visuals include player and enemy health bars, a stamina bar, and ability cooldown indicators, which are all fully implemented and visually polished. However, the menu systems, which are the pause, main, and the AI States Debug Menu, are purely functional and lack any aesthetic appeal or cohesion with the game's visual style. A key objective for us is to redesign all our menus to be visually engaging and thematically consistent, using dedicated art assets to create a more enjoyable and polished user experience.

We plan to expand the number of menus to include essential interfaces such as a settings/options menu, a player class selection screen, and potentially a map selection screen. The most significant overhaul is reserved for the AI States Debug Menu. Currently, it functions as a pause-screen tool for inspecting and altering the state of a single NPC. Some of the future improvements will include:

- The ability to select and monitor multiple NPCs simultaneously.
- The menu will remain open while the game is running, allowing developers to observe state transitions dynamically.
- A pause mode will let the user freeze the game and change the current state of all selected NPCs.

Map Development

Map design has received limited focus thus far, and while present in the game, it lacks key components such as collision and full map expansion. The current implementation includes sufficient detail to create a distinct and engaging environment, but remains incomplete. Moving forward, implementing collision is the highest priority, with completing the remaining portion of the map as a close second. Once the collision is implemented, additional details may be added as time permits. This means that if a capstone member finishes their work for a sprint early, they may work on the map. The additional detailing of the map includes adding trees, rocks, paths, and other smaller details that fill the empty spaces.

MoSCoW and Sprint Timeline

Both the MoSCoW analysis and timeline of each sprint can be seen in Appendices A and B, respectively. When creating the sprint timeline, we opted to have the earlier sprints more dense with work as opposed to the final few sprints due to the workload each team member will likely have as the semester progresses. The first few sprints are delegated for researching new systems our team may use and refactoring some of the foundation of the game. The second half of the semester will be focused on preparing for the poster session and presentation, alongside debugging and adding environmental effects to the game.

Conclusion

In this game, our goal is to create complex enemies that use different models for their decision-making process, as well as algorithms to gracefully move around the map. We want the player to have a variety of ways to take on the game's challenges in the form of different classes that can interact with enemies in different ways. By the end of this project, we plan to

have more in-depth enemy AI than what we had from the previous semester, and to be able to visualize the enemy AI as it moves from one state or behavior to another. This project is something our team is passionate about, and it represents an opportunity to apply and expand upon the skills and concepts we have developed throughout our time as students.

Works Cited

- Barriaes, Sergio Ocio. "You Had Me at 'AAAAHHH' – On the Importance of Reactions in Game AI." GameAIPro, Dec. 2021,
www.gameapro.com/GameAIProOnlineEdition2021/GameAIProOnlineEdition2021_Chapter11_You_had_me_at_AAAAHHH_On_the_importance_of_reactions_in_game_AI.pdf.
- "Navigation Meshes and Pathfinding." GameDev.net,
www.gamedev.net/tutorials/programming/artificial-intelligence/navigation-meshes-and-pathfinding-r4880/.
- Normoyle, Aline, et al. *The Curation Tree: A Lightweight Behavior Tree Framework for Implementing Puzzle and Narrative Games* | Proceedings of the 19th International Conference on the Foundations of Digital Games, doi.org/10.1145/3649921.3659840.
- Nystrom, Robert. "Game Programming Patterns." State · Design Patterns Revisited · Game Programming Patterns, gameprogrammingpatterns.com/state.html.
- Wouter van Toll, Atlas F. Cook IV, Roland Geraerts. Navigation Meshes for Realistic Multi-Layered Environments. webpace.science.uu.nl/~gerae101/pdf/navmesh.pdf.

Appendix A: MoSCoW Analysis

Must Have:

- Hierarchical state machine for enemies
- Behavior tree for enemies
- An enemy state that alters pathfinding (such as an enemy in the Burning state running around randomly instead of moving towards or away from the player)
- Player-inflicted states (such as the player inflicting a Burning state onto the enemy)
- Functional enemy pathfinding
- A GUI that shows the states/behaviors of the enemy (can visualize the state machines and behavior trees)
- Enemies with new enemy AI (such as the teleporting “assassin” enemy)
- Boss with behavior trees

Should Have:

- Integrate Navigation Mesh into the current pathfinding algorithm
- Enemy-inflicted states (such as the enemies sending the player into a Stunned state)
- Environmental interactions
- Visualize player states
- Ranged player class
- Third player class
- Settings menu

Could Have:

- Fluid animations
- Music and sound effects
- Class selection menu
- Player experience
- Player statistics
- In-game checkpoints

Won't Have:

- Save game progress
- Save user credentials
- Login menu
- Prevent piracy

Appendix B: Sprint Schedule

Sprint 1: 02/06/2026

- Research how to implement a hierarchical state machine
- Research the future development of a behavior tree for the first boss
- Create test enemies for class-based FSM and behavior tree
- Research the use of Navigation Mesh for enemy pathfinding
- Create a test environment for Navigation Mesh
- Work on bug fixes from the previous semester regarding pathfinding, enemy Retreat state, healer enemy, and AI states menu
- Art asset collection for the second player class and new enemies

Sprint 2: 02/20/2026

- Refactor the enemy finite state machine (FSM) to have a class structure with enemy state logic stored in separate classes
- Make scaffolding of the behavior tree, including different node types that will be used by the first boss
- Integration of Navigation Mesh into the game
- Refactor the player class to act as a parent class for future player classes
- Refactor player abilities to differentiate between ranged and melee attacks
- Develop a ranged player class
- Develop Bleed and Stun States that the enemies can enter

Sprint 3: 03/06/2026

- Complete development of behavior tree nodes for the first boss
- Create a “slime” enemy AI
- Begin work on “necromancer” boss’s behavior trees
- Develop the ability for enemies to be resurrected
- Add bleeding and stunning abilities to the current roster of enemies
- Begin work on visualizing a behavior tree

Sprint 4: 03/20/2026

- Poster preparations
- Finish development of behavior trees for the “necromancer” boss
- Allow for the boss's projectiles to split once collided with a surface
- Expand the dungeon map to include the boss arena
- Add the ability for the player to mitigate or avoid damage
- Create a “bruiser” enemy AI if time permits

- Continue work on visualizing enemy states and behaviors

Sprint 5: 04/03/2026

- Create an “assassin” enemy AI
- Allow for the player to movement cancel abilities (cancel ability animation by moving)
- Allow for the player to regain health
- Add controller support to the game
- Continue work on first boss if needed

Sprint 6: 04/17/2026

- Polish and populate the map
- Develop a Burning state that enemies can enter
- Add third “pyromancer” player class
- Continue work on player abilities if needed
- Continue work on basic enemies if needed

Sprint 7: 05/01/2026

- Outline of what we are gonna talk about.
- Finish work on map
- Finish work on enemy state and behavior visualization
- Allow for fire to spread across map tiles
- Continue work on Burning state and third player class if needed

Sprint 8: 05/15/2026

- Presentation preparations
- Player and enemy stat balancing
- Bug fixing